Interview Task: Job Queue Service

Welcome

Thank you for participating in our technical assessment process. We're excited to see what you build!

This project is designed to evaluate some of your skills that are required in the position you are candidate for.

Take your time, think through the edge cases, and show us how you approach complex problems.

About This Assessment

You'll be building a distributed background job processing system. This is a core infrastructure component used in many production systems for handling async tasks like sending emails, processing uploads, generating reports, and more.

We're interested in seeing how you handle concurrency, failure scenarios, and state management. There's no single "right" answer — we want to see your thought process and trade-offs.

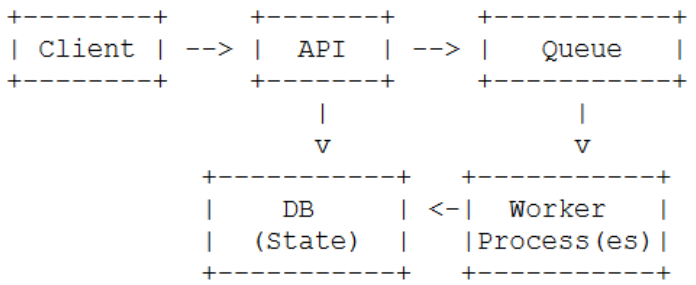
Using AI Tools

You are explicitly permitted and encouraged to use AI coding assistants during this assessment.

System Overview

Build a job queue system with these components:

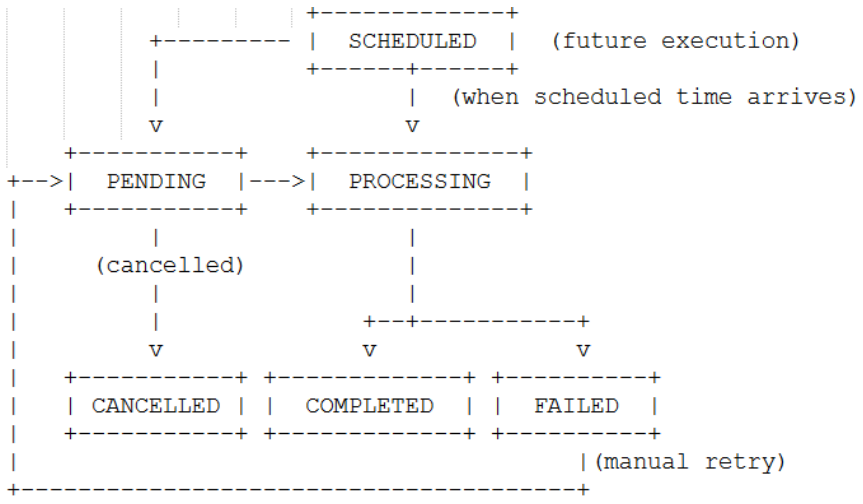
1. API Service — Accepts job submissions and status queries
2. Queue — Holds pending jobs (use any queue/cache technology)
3. Worker Process(es) — Pull jobs and execute them
4. Database — Stores job state and results



Functional Requirements

API Endpoints

- Submit a new job
- Get job status, result, or error
- List jobs with filters (status, type)
- Cancel a pending/scheduled job
- Retry a failed job
- Health check with queue statistics
- Job Lifecycle



Job Types to Implement

Implement these mock job types to demonstrate your system:

1. Email Job

Simulates sending an email. Sleep 1–3 seconds, return success with mock message ID.

2. Webhook Job

Simulates calling an external webhook. Sleep 1–2 seconds. 80% success rate, 20% simulated failure (for testing retry logic).

3. Aircraft transit report

Simulates generating an aircraft transit between any 2 airports report with dates and geo locations of the origin and the destination. In the UI it should present a map with the path of the aircraft. You can use any free public API or makeup the data.

Bonus: present hover capabilities that will display aircraft and transit details.

4. Batch Job

Processes multiple items. Process each item with small delay, track progress percentage, return summary.

Data Model

Design your data model to support:

Job

- Unique identifier
- Job type and payload (JSON)
- Status (scheduled , pending , processing , completed , failed , cancelled)
- Priority level (higher = more urgent)
- Attempt tracking (current attempts, max attempts)
- Error information for failed jobs
- Progress percentage (for batch jobs)
- Scheduling (optional future execution time)
- Timestamps (created, started, completed)
- Result storage (JSON)
- Idempotency key (for duplicate prevention)

Job Log (optional but recommended)

- Associated job
- Log level (info, warning, error)
- Message and metadata
- Timestamp

Critical Implementation Details

1. Job Pickup — Preventing Duplicates
2. Worker Crash Recovery
3. Retry with Backoff
4. Priority Queue
5. Scheduled Jobs
6. Idempotency

Document your approach in DECISIONS.md

Submission Requirements

Submit a Git repository containing:

```
your-project/
├── README.md
├── AI_USAGE.md
├── DECISIONS.md
├── docker-compose.yml
├── Dockerfile
├── (your application code)
└── (your tests)
```

README.md must include:

1. How to run the project
2. How to run tests
3. How to submit a test job (example request)
4. Brief architecture overview

DECISIONS.md (Required)

```
# Design Decisions

## 1. Job Pickup Strategy

**Approach chosen:** [Your approach]

**Why:** [Your reasoning]

**Trade-offs:** [What you gave up, what you gained]

---

## 2. Worker Crash Recovery

**Approach chosen:** [Your approach]

**Why:** [Your reasoning]

**What happens if worker crashes mid-job:** [Explain the recovery flow]

---

## 3. Priority Queue Implementation

**Approach chosen:** [Your approach]

**Why:** [Your reasoning]

---

## 4. Retry Backoff Strategy

**Approach chosen:** [Your approach]

**Timing:** [Specific delays you chose]

---

## 5. One Thing I Would Do Differently With More Time

[Be honest — what did you skip or simplify?]
```

AI_USAGE.md

AI Tool Usage

Tools I Used

[List the AI tools you used]

What Helped Most

[Describe 1-2 specific cases where AI helped significantly]

What I Had to Fix

[Describe 1-2 cases where AI gave incorrect advice — especially around concurrency]

What AI Struggled With

[Any parts where AI wasn't helpful]

Text / MD file/s containing your chat with the AI tool

Tips for Success

1. Get the basic flow working first: submit → queue → process → complete
2. Add failure handling second: retries, error storage
3. Make sure the UI is clean.
4. Tackle concurrency.
5. Handle the map and the API part.
6. Be honest in DECISIONS.md: we value self-awareness over perfection
7. Test the worker independently: it's easy to miss bugs there
8. Try to make small and clean commits, don’t push everything at once.